

Generic Methods Übungen

Random Element

Erstelle eine Klasse **RandomUtil**.

Implementiere eine **generische, statische** Methode **randomElement**.

Die Methode gibt ein **zufälliges Element aus Array** zurück. Das Array wird als generischer Parameter übergeben, dessen Elemente von Typ **T** sind.

Schreibe einen **Unit Test** für die Methode. Nutze für deinen Testfall die Annotation **@RepeatedTest(5)** statt **@Test**.

Random Element: Lösung

```
final class RandomUtil {  
  
    private static final Random RANDOM = new Random();  
  
    static <T> T randomElement(T[] elements) {  
        int randomIndex = RANDOM.nextInt(elements.length);  
        return elements[randomIndex];  
    }  
}
```

```
class RandomUtilTest {  
  
    @RepeatedTest(5)  
    void randomElement() {  
        var elements = new Integer[]{1, 2, 3, 4, 5};  
  
        var actual = RandomUtil.randomElement(elements);  
  
        assertTrue(Arrays.asList(elements).contains(actual));  
    }  
}
```

Übrigens: Eine alternative Lösung wäre, die Klasse Random per Dependency Injection oder Mockito zu mocken und dadurch im Testfall zu erzwingen, dass die Methode randomElement(T[] elements) immer ein bestimmtes Element aus dem Array zurückgibt, beispielsweise immer das Element an der Stelle (Index) 2.

Runnable

Erstelle eine Klasse **CollectionUtil**.

Implementiere die **statische, generische** Methode **void runAll**.

Die Methode erhält ein **Iterable** als Parameter, dessen Elemente durch **Runnable** beschränkt sind. Runnable ist ein vorhandenes Java-Interface.

Die Methode runAll führt **alle Elemente** im Iterable mit **run()** aus.

Schreibe einen **Unit Test** für die Methode.

Runnable: Lösung

```
final class CollectionUtil {  
  
    static void runAll(Iterable<? extends Runnable> runnables) {  
        runnables.forEach(Runnable::run);  
        //    for (Runnable runnable : runnables) {  
        //        runnable.run();  
        //    }  
    }  
  
    public static void main(String[] args) {  
        Iterable<Runnable> runnables = List.of(  
            () -> System.out.println("Hallo"),  
            () -> System.out.println("Welt!")  
        );  
        runAll(runnables);  
    }  
}
```

```
class CollectionUtilTest {  
  
    @Test  
    void runAllWithOneRunnable() {  
        int[] numberOfEvaluatedCallbacks = {0};  
        Iterable<Runnable> runnables = List.of(() -> numberOfEvaluatedCallbacks[0]++); // not pure lambda  
  
        CollectionUtil.runAll(runnables);  
  
        assertEquals(1, numberOfEvaluatedCallbacks[0]);  
    }  
  
    @Test  
    void runAllWithMultipleRunnables() {  
        int[] numberOfEvaluatedCallbacks = {0};  
        Runnable counter = () -> numberOfEvaluatedCallbacks[0]++; // not pure lambda  
        Iterable<Runnable> runnables = List.of(counter, counter, counter);  
  
        CollectionUtil.runAll(runnables);  
  
        assertEquals(3, numberOfEvaluatedCallbacks[0]);  
    }  
}
```

copyNumbers

Verwende die Klasse **CollectionUtil**.

Implementiere die Methode **static void copyNumbers**, die **alle Elemente aus source nach target kopiert**.

Die Methode erhält zwei Parameter **source** und **target**. Beide Parameter sind von Typ **Collection** und sollen **auf Number beschränkt** sein. Number ist eine vorhandene Java-Klasse.

Implementiere die Methode einmal nur **mit Wildcards** und einmal **ohne Wildcards**.

Was ist der **Unterschied zwischen beiden Varianten**?

Schreibe einen **Unit Test**, der beide Varianten testet.

copyNumbers: Lösung

```
final class CollectionUtil {
```

```
    static void copyNumbersWildcards(Collection<? extends Number> source, Collection<? super Number> target) {  
        target.addAll(source);  
    }
```

```
    static <T extends Number> void copyNumbersGeneric(Collection<T> source, Collection<T> target) {  
        target.addAll(source);  
    }  
}
```

Variante Wildcards:

- source kann nur gelesen werden und target nur geschrieben.
- source und target können unterschiedliche Typen enthalten.
- target muss von Typ Number sein: `copyNumbers(source, target);` // *target ist List<Number>.*

Variante Generic Method

- Beide Liste müssen dieselben Typen enthalten:
`copyNumbers(source, target);` // *wenn source ein List<Integer> ist, muss target auch ein List<Integer> sein.*
- source und target können sowohl gelesen als auch geschrieben werden.

```
public static void main(String[] args) {  
    List<Integer> integers = new ArrayList<>(List.of(1, 2, 3));  
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f));  
    copyNumbersWildcards(integers, numbers);  
    copyNumbersGeneric(integers, numbers); // does not compile  
}
```

Wildcards: extends vs super

```
no usages
static void copyNumbersWithWildcards(Collection<? extends Number> source, Collection<? extends Number> target) {
    for (Number elem : source) {
        target.add(elem);
    }
    System.out.println(target);
}
```

Frage: Warum funktioniert der Code nicht?

Angenommen source ist ein `ArrayList<Double>` und target ist ein `ArrayList<Integer>`, d. h. diese Typen werden beim Methodenaufruf von `copyNumbersWithWildcards` übergeben.

Sowohl `Double` als auch `Integer` extends `Number`, aber ein `Double` (source) kann nicht zu einer `List<Integer>` (target) hinzugefügt werden – `Double` passt nicht in die `Integer`-Liste!

Mit `Collection<? super Number> target` stellt der Compiler sicher, dass `ArrayList<Double>`, `ArrayList<Integer>`, `ArrayList<Long>`, usw. nicht als target übergeben werden dürfen.

Es muss stattdessen mindestens ein `ArrayList<Number>` als target übergeben werden oder etwas "darüber", z. B. `ArrayList<Object>` – in solchen Listen passen problemlos `Double`, `Integer`, `Long`, usw. und können hinzugefügt werden.

copyNumbers: Lösung (Unit Test)

@Test

```
void copyNumbersWildcards() {  
    List<Integer> integers = new ArrayList<>(List.of(1, 2, 3));  
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f));  
  
    CollectionUtil.copyNumbersWildcards(integers, numbers);  
  
    List<Number> expected = List.of(1.1, 2, 3.3f, 1, 2, 3);  
    assertEquals(expected, numbers);  
}
```

@Test

```
void copyNumbersGeneric() {  
    List<Integer> integers1 = new ArrayList<>(List.of(1, 2, 3));  
    List<Integer> integers2 = new ArrayList<>(List.of(4, 5, 6));  
  
    CollectionUtil.copyNumbersGeneric(integers1, integers2);  
  
    List<Number> expected = List.of(4, 5, 6, 1, 2, 3);  
    assertEquals(expected, integers2);  
}
```

findMax

Verwende die Klasse **CollectionUtil**.

Implementiere die **statische, generische** Methode **T findMax**.

Die Methode erhält eine **List** mit Elementen von Typ **T** als Parameter und gibt einen Wert von Typ **T** zurück.

Typ **T** ist durch **Comparable<T>** beschränkt.

Falls die übergebene **Liste keine Elemente enthält**, wird eine **NoSuchElementException** geworfen.

Die Methode **findet** das **größte Elemente** in der übergebenen Liste und gibt das Element **als Rückgabewert** zurück.

Schreibe einen **Unit Test** für die Methode.

Hinweis:

Die Klassen **Number** und **String** implementieren in Java bereits das Interface **Comparable**.
D. h. du kannst die Methode **findMax** mit **Numbers** oder **Strings** testen.

findMax: Lösung

```
static <T extends Comparable<T>> T findMax(List<T> comparables) {
    return comparables.stream()
        .max(Comparable::compareTo)
        // .max((a, b) -> a.compareTo(b))
        .orElseThrow();
}

static <T extends Comparable<T>> T findMaxWithoutStream(List<T> comparables) {
    if (comparables.isEmpty()) {
        throw new NoSuchElementException();
    }

    T max = comparables.getFirst();
    for (T element : comparables) {
        if (element.compareTo(max) > 0) {
            max = element;
        }
    }
    return max;
}
```

```
class CollectionUtilTest {

    @Test
    void findMaxInteger() {
        List<Integer> integers = List.of(1, 2, 3);

        var actual = CollectionUtil.findMax(integers);

        assertEquals(3, actual);
    }

    @Test
    void findMaxString() {
        List<String> strings = List.of("a", "b", "c");

        var actual = CollectionUtil.findMaxWithoutStream(strings);

        assertEquals("c", actual);
    }
}
```